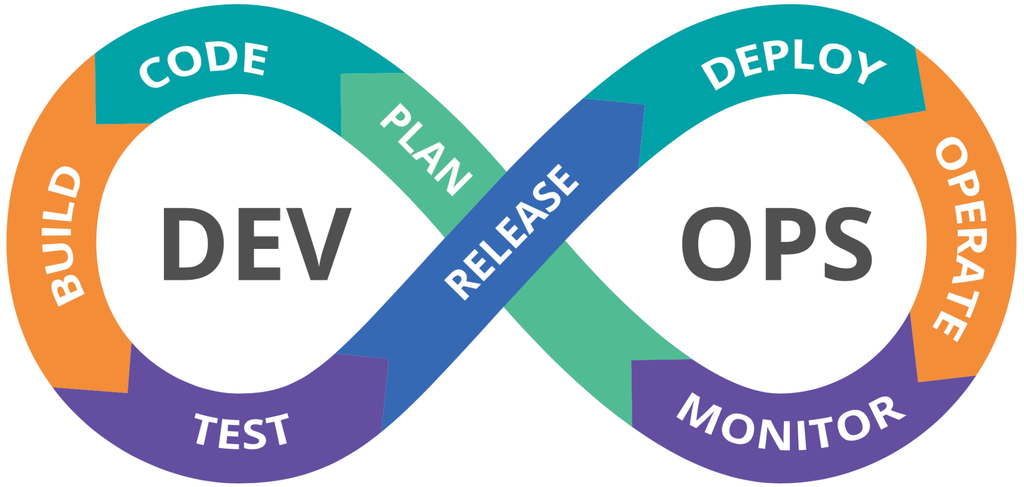
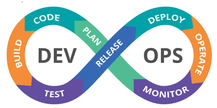


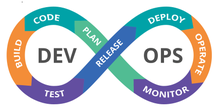
Tehničke prakse za uvođenje principa toka

Primena arhitektura koje obezbeđuju nizak rizik pri isporuci



Tehničke prakse za uvođenje principa toka

- Kontinualna isporuka (*Continuous Delivery*) podrazumeva
  - uspostavljanje *deployment pipeline-a*
  - omogućavanje brzog i pouzdanog automatizovanog testiranja
  - omogućavanje i praktikovanje kontinualne integracije
  - **automatizaciju i upravljanje razvojem koji omogućava isporuku uz male rizike (*low risk deployment*)**



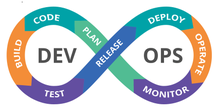
Problem podobne arhitekture

- Skoro sve velike kompanije su imale probleme sa svojim aplikacijama izazvane njihovom arhitekturom koja vremenom više nije bila prikladna za uslove korišćenja

- Arhitektura kritičnih poslovnih aplikacija vremenom migrira na novija rešenja, bolje prilagođena tekućim realnim uslovima korišćenja - evolutivni princip arhitekture softverskih sistema

- Prerana primena arhitekture koja nije pouzdana takođe nije poželjno stanje

- Izazov je uvek kako obezbediti migraciju na nove arhitekture - od one koje trenutno imamo, ka onoj koju stvarno u datom momentu (i predvidljivoj budućnosti) trebamo

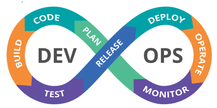


Problem podobne arhitekture

- Često se pri ovoj evolutivnoj promeni koristi princip uvođenja novih rešenja na one delove aplikacija kod kojih se očekuje najveći pozitivan efekat i najbolji povrat novca investiranog u razvoj.

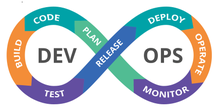
- “strangler application” princip - postepenog nadopunjavanja i zamene starih rešenja servisnim komponentama - obezbeđuje nedisruptivan postepeni prelazak na nova rešenja i arhitekturu

- suštinski za velike aplikacije, koje treba da pruže visok nivo skalabilnosti i otpornosti na otkaze, prelazak sa monolitnih aplikacija na servisno orijentisane arhitekture se pokazuje kao pozitivan razvojni put



Tipovi arhitektura

	Pros	Cons
Monolithic v1 (All functionality in one application)	• Simple at first • Low inter-process latencies • Single codebase, one deployment unit • Resource-efficient at small scales	• Coordination overhead increases as team grows • Poor enforcement of modularity • Poor scaling • All-or-nothing deploy (downtime, failures) • Long build times
Monolithic v2 (Sets of monolithic tiers: "front end presentation," "application server," "database layer")	• Simple at first • Join queries are easy • Single schema, deployment • Resource-efficient at small scales	• Tendency for increased coupling over time • Poor scaling and redundancy (all or nothing, vertical only) • Difficult to tune properly • All-or-nothing schema management
Microservice (Modular, independent, graph relationship vs. tiers, isolated persistence)	• Each unit is simple • Independent scaling and performance • Independent testing and deployment • Can optimally tune performance (caching, replication, etc.)	• Many cooperating units • Many small repos • Requires more sophisticated tooling and dependency management • Network latencies

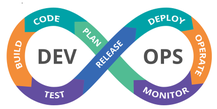


Arhitektura aplikacija koja obezbeđuje visoku produktivnost, mogućnost testiranja i bezbednost

- kod monolitnih arhitektura vremenom kompleksnost rešenja raste i postaje opterećenje za dalji razvoj jer zbog “čvrste spregnutosti” komponenti svaka nova, pa i mala izmena potencijalno izaziva probleme i ne može se više jednostavno “komitovati”

- razvoj servisnih modula omogućava da se komponente razlože na jasno definisane funkcionalne celine, koje često obavljaju vrlo mali skup, dobro definisanih funkcija i mogu se razvijati, ažurirati i isporučivati nezavisno

- kako svaki servis ima dobro definisan API to omogućava lakše testiranje i uspostavljanje SLA između različitih timova



Strangler application obrazac kao mehanizam za bezbedan proces evolucije

- kada imamo zastarelu aplikaciju, ili kada utvrdimo da je, za sadašnje potrebe i budući razvoj, aplikacija previše čvrsto spregnuta može se krenuti u postupak izdvajanja određenih funkcionalnosti u posebne (servisne) module

- na ovaj način razovjni timovi mogu da rade izolovano, testiraju nove module, i kada su novi moduli testirani, njihovom funkcionalnošću se zamenjuje istovetna funkcionalnost u starom sistemu.

- novi servisi se postepeno dodaju u sistem, a gde je to potrebno oni vrše interakciju sa starim sistemom time “skrivajući” staru funkcionalnost iza novih servisa - vremenom servisne komponente će preuzeti potpuno funkcionalnosti starog sistema - kada on postaje nepotreban